

NEXT JS

Better Fetching

Better fetching refers to the enhanced `fetch()` behavior in Next.js App Router, where data fetching is optimized with automatic caching, request deduplication, and streaming support.

This allows developers to fetch data directly in Server Components without using special hooks like `getServerSideProps`.

Example:

```
const res = await fetch("https://api.example.com/posts")
const posts = await res.json()
```

The Core Idea of Better Fetching

In older React apps, data fetching usually happened like this:

Browser loads page



React runs



`useEffect` runs



`fetch` API



data comes



UI updates

Problems:

- blank screen first
- slow
- extra network requests
- poor SEO

Next.js App Router changes this by allowing data fetching on the server during rendering.

So the flow becomes:

Request page



Server fetches data



Server renders UI



HTML sent to browser

The user sees ready content immediately.

component runs on the server, so the data is fetched before HTML reaches the browser.

That's the first improvement.

Next.js Upgrades the Native fetch

1. Automatic Caching

By default, Next.js caches fetch results.

If multiple users request the page, Next.js may reuse the cached data instead of hitting the API again.

Benefits:

- **faster pages**
- **fewer database/API calls**
- **better scalability**

2. Request Deduplication

If the same fetch happens multiple times during rendering:

```
await fetch("/api/posts")  
await fetch("/api/posts")
```

Next.js only makes one network request and shares the result.

This prevents wasted requests.

3. Built-in Revalidation

You can control when cached data should refresh.

Example:

```
fetch(url, {  
  next: { revalidate: 60 }  
})
```

Meaning:

cache data

↓

after 60 seconds

↓

refresh it

This is similar to ISR (Incremental Static Regeneration).

4. Dynamic Fetching (No Cache)

Sometimes you always want fresh data.

Example:

```
fetch(url, {  
  cache: "no-store"  
})
```

Now every request:

request page



fetch fresh data



render page

This behaves like server-side rendering (SSR).

5. Works with Streaming

Because fetching happens in async server components, it works with React Suspense.

The Big Mental Model

Better fetching connects four major Next.js features:

Server Components



Better Fetch



Caching + Revalidation



Streaming UI

Everything is designed so that data fetching happens closer to the server.

Server Actions

A server-side function that can be directly invoked from a React component without creating an API route.

Before Server Actions, the flow usually looked like this:

Client → API Route → Server Logic → Database

Example:

React Form



`fetch('/api/create-user')`



API Route



Prisma / DB

A lot of boilerplate.

With Server Actions

Now the flow becomes:

Client Component / Form



Server Action Function



Database

No API route needed.

You **directly call a server function from your UI.**

Core Idea

A **Server Action** is just a **server-side function** marked with:

"use server"

Example:

```
export async function createUser() {  
  "use server"  
  
  console.log("Runs on server")  
}
```

This function **never runs in the browser**.

Example 1 — The Simplest Server Action

app/actions.ts

```
export async function createUser() {  
  "use server"  
  
  console.log("User created")  
}
```

app/page.tsx

```
import { createUser } from "./actions";  
  
export default function Page() {  
  return (  
    <form action={createUser}>  
      <button>Create User</button>  
    </form>  
  );  
}
```

When button is clicked:

Browser



Next.js sends request automatically



Server Action runs



Page updates

No fetch. No API route.

Secret Magic Behind the Scenes

When you do:

<form action={createUser}>

Next.js secretly converts it into:

POST /_actions

Then runs the function on the server.

You don't see the API.

But it exists internally.

Why Server Actions Are Powerful

They remove a lot of boilerplate.

Instead of:

API Route

Controller

Validation

Fetch

You write:

Server Function

Boom.

When You Should Use Server Actions

Good for:

- ✓ Form submissions
 - ✓ Database mutations
 - ✓ Creating data
 - ✓ Updating data
 - ✓ Deleting data
-

When NOT to Use Them

Still use API routes for:

- Public APIs
- Mobile apps
- Third-party integrations
- Webhooks
- Sending Form Data

Most common real usage.

UI

```
<form action={createUser}>  
  <input name="username" />  
  <button>Submit</button>  
</form>
```

Server Action

```
export async function createUser(formData: FormData) {  
  "use server"  
  
  const username = formData.get("username")  
  
  console.log(username)  
}
```


What are Server Actions in Next.js?

Answer:

Server Actions are server-side functions that can be called directly from React components without creating API routes.

They are marked with "use server" and run only on the server.

They are mainly used for mutations like creating, updating, or deleting data.

Are Server Actions secure?

Answer:

Yes, because they run only on the server.

Sensitive logic like:

- database queries
- authentication
- API keys

remains on the server.

What are the limitations of Server Actions?

Answer:

Some limitations:

- Only work inside Next.js app router
- Not suitable for public APIs
- Cannot run on the client
- Must follow server/client component boundaries

What are the benefits of Server Actions?

Answer:

Main benefits:

- No API routes needed
- Less boilerplate

- **Better performance**
- **Built-in form handling**
- **Works with progressive enhancement**
- **Direct server access to database**

Can Server Actions be used in Client Components?

Answer:

Yes, but they must be imported from a server file.

Can Server Actions be used in Client Components?

Answer:

Yes, but they must be imported from a server file.

Next.js creates a network bridge automatically.

How do Server Actions receive data from forms?

Answer:

They receive a FormData object.

```
export async function createUser(formData: FormData) {  
  "use server"
```

```
  const name = formData.get("name")  
}
```

Streaming

1 The Problem Streaming Solves

Traditional server rendering works like this:

Request page



Server fetches ALL data



Server builds FULL HTML



Send HTML to browser

Example problem:

Navbar fetch → 50ms

Sidebar fetch → 100ms

Dashboard fetch → 2s

User waits 2 seconds for everything.

Even though Navbar was ready in 50ms.

That's bad UX.

2 Streaming: Send UI in Pieces

Streaming allows the server to send parts of the page as soon as they are ready.

Instead of:

Wait → Send whole page

It becomes:

Send Navbar

Send Sidebar

Send Dashboard later

So the browser starts rendering immediately.

Think of it like YouTube buffering video chunks.

3 Where Suspense Comes In

React needs to know:

“What should I show while this part is loading?”

That’s what Suspense does.

Example:

```
import { Suspense } from "react";
import Dashboard from "../Dashboard";

export default function Page() {
  return (
    <div>
      <Navbar />

      <Suspense fallback={<p>Loading dashboard...</p>}>
        <Dashboard />
      </Suspense>
    </div>
  );
}
```

Flow:

Page loads

Navbar renders immediately

Dashboard loading → show fallback

Dashboard ready → replace fallback

Next.js Special Feature: loading.tsx

Next.js automatically uses Suspense via loading.tsx.

Example folder:

app/dashboard/

page.tsx

loading.tsx

loading.tsx

```
export default function Loading() {  
  return <p>Loading dashboard...</p>;  
}
```

Next.js automatically wraps the page in Suspense.

Flow:

User opens /dashboard



Show loading.tsx



Page data loads



Real page replaces loading

Streaming with Data Fetching

Example server component:

```
async function Posts() {  
  const posts = await fetchPosts();  
  
  return (  
    <ul>  
      {posts.map(p => <li key={p.id}>{p.title}</li>)}  
    </ul>  
  );  
}
```

Page:

```
<Suspense fallback=<p>Loading posts...</p>>  
  <Posts />  
</Suspense>
```

Now Posts loads separately.

Streaming allows the server to send HTML to the browser in chunks instead of waiting for the entire page to render.

Suspense

Suspense lets React show a fallback UI while a component waits for asynchronous data.

What is React Suspense?

Answer:

React Suspense is a feature that allows components to wait for asynchronous operations (like data fetching) and show a fallback UI while the component is loading.

Example:

```
<Suspense fallback={<p>Loading...</p>}>  
  <Posts />  
</Suspense>
```

While Posts is loading, React displays the fallback UI.

How does Streaming work with Suspense?

Answer:

Suspense creates loading boundaries in the UI.

When a component inside Suspense is still loading, React sends the fallback UI first and streams the actual component later when it is ready.

Does Next.js support Streaming automatically?

Answer:

Yes. In the App Router, Next.js supports streaming automatically when a Suspense boundary exists.

This can be created by:

- **Using <Suspense>**

- Using a loading.tsx file

Why is Streaming important for performance?

Answer:

Streaming improves perceived performance because users can see parts of the UI earlier instead of waiting for the full page to load.

This reduces time to first meaningful paint.

What is the fallback in Suspense?

Answer:

Fallback is the temporary UI shown while the component inside Suspense is loading.

Can Suspense work with data fetching?

Answer:

Yes. In React Server Components and Next.js App Router, Suspense can work with async server components that fetch data.

React waits for the data and shows the fallback until the component is ready.

What happens if there is no Suspense boundary?

Answer:

If there is no Suspense boundary, React must wait for the entire component tree to finish rendering before sending HTML, which prevents streaming.

Edge Runtime vs Node.js Runtime (Next.js)

Node.js Runtime

Runs your code on a traditional server.

Characteristics

- Full Node.js APIs available
- Can use libraries like fs, crypto, Prisma, etc.
- Slower for global users because server may be far away
- Best for database-heavy logic

Example use cases:

- database queries
- authentication logic
- heavy backend processing

Edge Runtime

Runs your code on edge servers close to the user (CDN locations).

Characteristics

- Extremely low latency
- Runs near the user geographically
- Limited APIs (not full Node.js)
- Uses Web APIs instead of Node APIs

Limitations:

- no fs
- no native Node libraries
- limited packages

Node runtime → full backend power

Edge runtime → super fast global response

Node runtime provides full Node.js capabilities for backend logic, while Edge runtime runs lightweight code closer to users for faster response times but with limited APIs.